Latest updates: https://dl.acm.org/doi/10.1145/2948708.2948715

RESEARCH-ARTICLE

DownChord and UpChord: A New Style of Keyboard Shortcuts based on Simultaneous Key-down and Key-up Events

OSCAR KIN-CHUNG AU, City University of Hong Kong, Hong Kong, Hong Kong

KIRA YIP-WO YEUNG, City University of Hong Kong, Hong Kong, Hong Kong

JACKY KIT CHEUNG, City University of Hong Kong, Hong Kong, Hong Kong

Open Access Support provided by:

City University of Hong Kong

PDF Download
2948708.2948715.pdf
07 March 2026
Total Citations: 3
Total Downloads: 208

Published: 08 May 2016

Citation in BibTeX format

ChineseCHI2016: The Fourth International Symposium of Chinese CHI
May 8, 2016
San Jose, USA

Chinese CHI '16: Proceedings of the Fourth International Symposium of Chinese CHI (May 2016)
https://doi.org/10.1145/2948708.2948715
ISBN: 9781450347600

DownChord and UpChord: A New Style of Keyboard Shortcuts based on Simultaneous Key-down and Key-up Events

Oscar Kin-Chung Au
City University of Hong Kong
Hong Kong
kincau@cityu.edu.hk

Kira Yip-Wo Yeung
City University of Hong Kong
Hong Kong
y@acg.hk

Jacky Kit Cheung
City University of Hong Kong
Hong Kong
ckitsa@gmail.com

ABSTRACT

Typical typing, as well as many other keyboard operations including using modifier-based shortcuts, is dependent upon the order of key press and avoids simultaneous key pressing. This paper introduces DownChord and UpChord, novel keyboard shortcut techniques, which utilizes the simultaneous key pressing and releasing for defining new classes of shortcuts. Our technique can be used on ordinary unmodified physical keyboard seamlessly – without interfering major functionality of computer keyboard such as typing and activation of existing modifier-based shortcuts. We conducted controlled experiments to determine the appropriate time thresholds for detecting shortcuts from the input keyboard event queue, which allows our shortcuts to be activated effortlessly with only average latency of 160 ms. Results of our user study show that DownChord and UpChord are promising shortcut techniques which gives competitive performance as existing modifier-base shortcuts.

CCS Concepts

•**Human-centered computing** → **Keyboards; Interaction techniques;**

Keywords

Physical Keyboard; keyboard shortcuts; modeless interaction.

1. INTRODUCTION

Keyboard shortcuts are efficient and direct interface for many interaction tasks in typical and traditional computer environment. On regular computer keyboards, the most common type of keyboard shortcuts are modifier-based shortcuts that the shortcuts are consists of one or more modifier keys and one regular key. To activate such shortcuts, the modifier keys have to be pressed before the regular (non-modifier) key to avoid ambiguity. Using key chord input, i.e. typing multiple regular keys at the same time, with simple checking of concurrently key pressings may not be feasible to defining new shortcuts since the general typing would be recolonized as chord input incorrectly.

Recently, physical keyboards have been equipped with different sensors for additional interactions such as touch-display [7], pressure input [9] or motion sensing [13], however expanding of the input space of the ordinary unmodified keyboard has not been focused much in prior work. Recently [17] gesture detection technique, GestKeyboard, had been introduced which allows additional complex gestures to be recognized with unmodified keyboard. The technique extends the design space of keyboard input however the gesture input still requires long input key sequence (10.2 keys in average) and long input time (>700ms in average).

We intend to enhance the shortcut design space of regular keyboard with efficient and simple input without any additional physical modification. In this paper, we present the DownChord (DC) and UpChord (UC) shortcuts, the novel keyboard shortcut techniques, which utilize the simultaneous key pressing and releasing for defining two new classes of keyboard shortcuts. Our techniques can be used on ordinary unmodified physical keyboard seamlessly and requires no additional hardware. It is highly integrated and allows seamless activation of the new shortcuts without interfering the major functionality of computer keyboard such as typing and activation of existing modifier-based shortcuts, and potentially reduces homing effort, such that users can complete more tasks without moving their fingers to access other modifier keys and functional keys, or using other input devices. Our basic idea is straightforward, by tracking the key press and release events as an event sequence, we can detect any simultaneous key pressing and releasing and recognize for shortcut activations.

In general, our technique has the following nice properties:

Simple activation – The shortcuts only involve a few adjacent keys and simple pressing and releasing motion which could be activated very fast as regular typing.

Large design space – The design space of keyboard shortcuts is greatly extended. New shortcuts can be defined based on the key combinations and locations on keyboard.

Compatible to typical typing – Our technique is based on simultaneous pressing or releasing of adjacent keys, which are in general not appear in typical typing and therefore our technique can be seamlessly integrated.

Compatible to modifier-based shortcuts – Our technique can work with existing modifier-based shortcuts, without causing any ambiguity. Moreover new shortcuts can be defined with both modifier keys and our technique, e.g. the DownChord shortcut Ctrl+O+P is activated by first pressing the Control key and then pressing O and P keys simultaneously.

Adaptable to different physical form – Our technique can be

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

Chinese CHI 2016 May 8, 2016, San Jose, USA

© 2016 ACM. ISBN 123-4567-24-567/08/06…$15.00

DOI: 10.1145/2948708.2948715

operated equally well on keyboards with different keycap profiles and key travel distances, since it only involves general key pressing and releasing but not sliding motion as required in the interfaces in [16] and [17].

2. RELATED WORK AND BACKGROUND

Besides typical typing, using sequence of key presses to represent trajectory input with physical keyboard had been introduced previously, for example in [11] an input method which simulates character shapes on numeric keypad for text input was presented. EdgeWrite [15] uses a smaller input scheme with only 4 keys to capture key sequences for text entry. GestKeyboard [17] adopts the whole keyboard to allow more complicated gesture input with key sequence. The input scheme or the gesture recognition algorithm of all these methods depend on the key pressing order. They share the same input space of sequent key input of typical typing, and therefore may cause ambiguity if no mode switching mechanism is used.

Recently physical keyboards have been augmented with additional hardware for supporting new interactions [6, 7, 9, 13]. In [9] additional press sensors were installed in the keyboard to provide individual analog press sensing for different keys. Touch display [7] allows applications to redefine keyboard layout and functions with the capacitive touch sensors and projector display augmented to keycaps. Actuated Keys [6] uses mechanically raising keycaps to provide physical feedback and extra interactions such as pushing and squeezing of the raised keys. In [13] the keyboard is augmented with infrared proximity sensors to provide motion gesture input on or above the device. All these works require additional and special hardware for providing extra input dimensions.

Chorded keyboards, such as MicroWriter [3], CyKey [1] and Twiddler [4] are kind of special input device that allows user to enter commands of characters by pressing several keys together, like playing a chord on a piano. With the large number of key combinations available, these devices allow text or commands to be entered with small number of keys. Most of these input devices were introduced as replacements of regular keyboards for mobile or single hand input, and usually their input methods consider only simple typing order and simultaneously typing. In contrast, our DC and UC shortcuts techniques aim at extending the input space of existing regular unmodified keyboards, by considering the key pressing and releasing as different operations for shortcuts and command activations.

Extensive work also has been conducted on using gesture [16, 10] or multitouch input [8] for text entry on touchscreen. Also there is existing work that detects hands and fingers, as well as finger chords [5, 14] on multitouch input devices for command and menu selection. All these works use the finger tapping and motion (i.e. path of touch point sequence) for the detection and recognition, but does not consider the touching and releasing individually.

The common design of physical keyboards consists of matrix of circuits such that keycaps are placed at the intersections of the column and row wires of the matrix [2]. Key presses cause the corresponding column and row wires be connected and thus the location of key press can be detected by checking the connectivity between the column wires and row wires. However, this simple design cannot detect key presses correctly when keys on multiple columns and rows (usually more than three keys) were pressed, because the pressed keys short circuit the involved multiple rows and columns such that incorrect key may be reported (a.k.a. the ghosting problem) or pressed keys were suppressed (a.k.a. the masking problem). In order to let our DC and UC shortcuts to be usable for most physical keyboards, we design the shortcuts to be input with single-hand input (using the fingers except thumb) and require all the shortcut keys on the same row.

The QWERTY layout [12] is the most common modern-day keyboard layout for Latin script and originally it was designed for preventing clash and jam of typebars of mechanical typewriters caused by neighbouring arms were pressed at the same time or in rapid succession. Its layout places commonly used letter pairs so that they were not neighbouring, thus reduces the chances of neighbouring keys to be pressed successively, also it encourages alternation between the hands during typing which potentially speeds up typing. The design of QWERTY layout in fact leaves the rooms of interaction with simultaneous key pressing and releasing of neighbouring keys, and our shortcut technique utilizes this to enhance the input space of regular keyboards.

3. DESIGN AND IMPLEMENTATION

In this section, we first give an overview design of DC and UC shortcut techniques and then the implementation details.

Common existing keyboard shortcuts mostly consist of one or more modifier keys (such as control, alternate or shift keys) and a regular key (e.g. alphabet or function keys). For example, the copy and paste shortcuts on Windows platform are Ctrl+C and Ctrl+X. Note that for these modifier-based shortcuts, the modifier key(s) has to be pressed first and kept pressing while the regular key is pressed, as illustrated in Figure 1b. The order cannot be reversed because OS cannot determine the pressed key is part of combination key of a modifier-based shortcut when the regular key is pressed first.

During typical typing, people are required to input characters in a predefined order according to the words, sentences or data needed to input. As a simple typing sequence of the word "city" illustrated in Figure 1a, keys are pressed with clear order and there is long enough time between any two successive key-down events such that the user can ensure the keys are input with correct order. Note that in typical typing the key-up events usually also in the order same as those of the key-down events, it is because in general the pressing time of each key are similar for the same computer user. Also note that the pressing periods of two sequent key presses may be overlapped, this is the reason why simple concurrent key press checking may not feasible for recognizing key-chord input.

Our basic idea is to extend the design space of keyboard shortcuts for enriching interaction of regular keyboard, by considering the simultaneous key press and release events as the unused user input pattern. With the observation that there are rare input cases that need keys to be pressed or released at the same time, in the paper we introduce the concept of having shortcuts to be activated when multiple keys are pressed or released almost *simultaneously*, rather than only using a fixed and determinable of key pressing order to define the keyboard shortcuts. We call our new technique as *DownChord* and *UpChord* shortcuts. The new shortcuts were designed to be input by one hand and the keys involved should be on the same row on the keyboard, as it is the simplest way to press multiple keys with the usual hand pose. The new shortcuts can be used together with typical typing and existing modifier-based shortcuts without any undesired ambiguity. We believe our proposed shortcut techniques are easy to learn and master and can be directly adopted for extending the set of shortcut commands, which is useful and valuable for many applications with large set of predefined operations, by considering using the DC and UC keyboard shortcuts could be in general faster than using other UI such as selection menu items or gestures inputs.

A DC shortcut is activated by pressing adjacent keys (2 to 4 keys) of the same row of the keyboard (e.g. the combination of A-S-D keys, as illustrated in Figure 1c) almost simultaneously. All

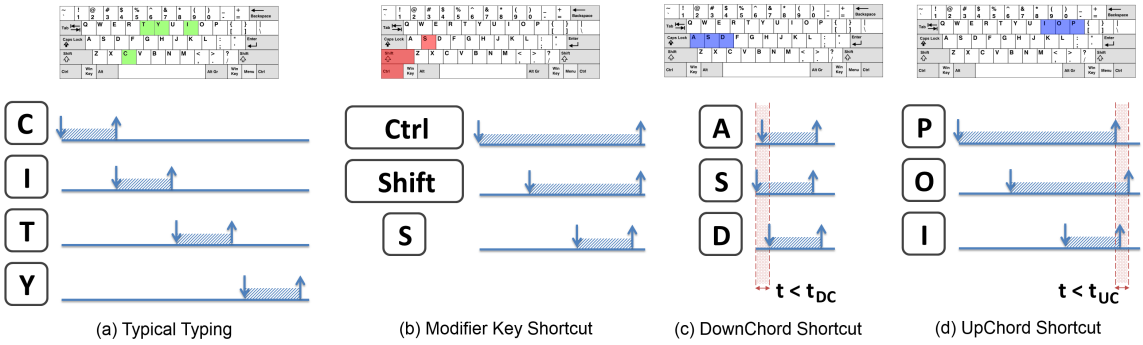

Figure 1: The illustrations of different typing scenarios. (a) Typical typing, (b) modifier-based shortcut, (c) DownChord shortcut and (d) UpChord shortcut.

key-down events of the key pressings should occur within a predefined short time threshold t_{DC}. Note that after activating the DC shortcut by simultaneous pressing, user tends to release the keys also almost simultaneously just like pressing the keys as a combined single button. In order to utilize the simultaneous key release events as recognizable UC shortcuts other than the DC shortcuts, we design the UC shortcuts that are activated by pressing the adjacent keys on the same row of keyboard non-simultaneously and releasing the keys almost simultaneously (e.g. the combination of P-O-I keys, as illustrated in Figure 1d), with the motion similar to drum one's fingers on table. It means that all key-up events of the key pressing should occur within a predefined short time threshold t_{UC}, while the time difference between the first and last key-down events should be greater than the given predefined time threshold t_{DC}. Note that in the activations of both DC and UC shortcuts, the exact key pressing order and the releasing order are not important.

3.1 Implementation

Once a DC or UC shortcut is recognized, the corresponding action associated with the shortcut will be performed, and the original key pressing events will be suppressed such that no character of the pressed keys will be send to the current process or window control. In order to support the key suppression, an event hook is used to capture and store all keyboard events in a queue, and the key-pressed events are dispatched to the current focused window control if and only if the events were not involved in any DC or UC shortcut activation. Listing 1 shows the pseudocode of the key event process algorithm. In order to dispatch the key-pressed events as soon as possible, three dispatch rules are introduced:

Rule 1: If two sequentially pressed keys were not of the same row or nearby columns, the key-pressed event for the first key will be dispatched.

Rule 2: A key-pressed event will be dispatched if the key is pressed for long enough time (t_{DK1}).

Rule 3: If the most recent key is pressed for long enough time (t_{DK2}), all key press events in the queue will be dispatched.

Listing 1: Pseudocode of the key event scanning algorithm.

```
Q = {}
while (true) {
  if (new KEYDOWN event e)
    Q.enqueuq(new Record(e.key, e.time));

  if (new KEYUP event e)
    if ((rec = Q.search(e.key)) != null)
      rec.upTime = e.time;

  if (CheckDownChord(Q) == true) {
    chord = new DownChord(Q);
    Q.clear();
    performAction(chord);
  }

  if (CheckUpChord(Q) == true) {
    chord = new UpChord(Q);
    Q.clear();
    performAction(chord);
  }

  // Dispatch rule 1
  if (Q.count() >= 2)
    if (Q.first.row != Q.second.row ||
         Q.first.column - Q.second.column >
             3)
      dispatchKeyPressed(Q.Dequeue());

  // Dispatch rule 2
  if (Q.count >= 1)
    if (currentTime - Q.first.downtime >
        t_DK1)
      dispatchKeyPressed(Q.Dequeue());

  // Dispatch rule 3
  while (Q.count >= 1 &&
         currentTime - Q.last.downtime >
             t_DK2)
    dispatchKeyPressed(Q.Dequeue());
}
```

Note that the time thresholds adopted should meet the conditions $t_{DK1} > t_{DK2} > t_{DC}$ and $t_{DK1} > t_{UC}$. These time thresholds are crucial to the usability and performance of our shortcut interface, since very short time thresholds will increase the rate of unsuccessful activations and lead to incorrect key inputs, while too long time thresholds will give undesired and observable typing lag. In the first experiment, we had conducted user study to estimate the suitable time thresholds for both the DC/UC shortcuts and the dispatch rules. We expect with the appropriate estimated thresholds, the DC and UC shortcuts can be integrated to applications with usual typing interaction. We had conducted another user study (Experiment 2) to demonstrate the compatibility of our shortcut interfaces by evaluating the unintended activations and delay time when our technique is adopted in actual typing scenarios. In order to measure the performance of our shortcut techniques, the performance evaluation (Experiment 3) was conducted by comparing our shortcut techniques with modifier-based shortcut interface.

4. EXPERIMENT 1 – TIME THRESHOLD STUDY

Figure 2: Pressing multiple keys on a row with up to 4-fingers of a hand.

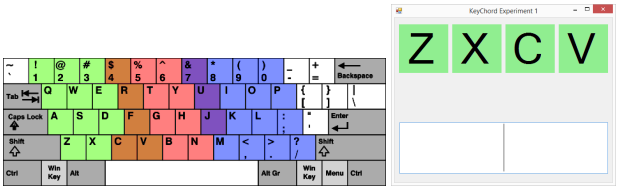

Figure 3: (Left) Shortcut locations defined for Experiment 1. Left, middle, and right regions are highlighted in green, red and blue respectively, overlapped regions are shown in orange and purple colors. (Right) Screen capture of the testing program used for Experiment 1.

In this experiment we conducted user study to evaluate the time performance of the DC and UC shortcut interfaces, with different shortcut settings (namely, the location and the number of keys involved), as well as two common switch techniques of computer keyboards. According to the results of the user study, we then suggest the appropriate time threshold settings such that most users can accurately activate and use our shortcuts with the suggested time thresholds, without undesirable long system delay.

Since it is possible to press multiple adjacent keys of a row with up to 4 fingers of a hand (see Figure 2), in general our shortcut techniques can be used with 2 to 4 fingers depending on the shortcut design. We believe different time threshold settings should be used for shortcuts of different number of keys, as in general users require more time when they are pressing or releasing more keys, even though they want to press or release the keys simultaneously. Therefore, in the user study we also include the factor of the number of keys being used, and suggest different appropriate time thresholds for DC and UC shortcuts of different number of keys.

The key response and travel distance of different keyboards vary from each other, and this mainly depends on the switch technique that the keyboard uses. In order to examine how different switch techniques affect the performance and accuracy of the activation of shortcuts, we consider the switch techniques as one of the factor in this experiment and have tested three keyboards (two scissor-switch membrane keyboards and one mechanical keyboard) with different travel distances.

The location of the shortcut keys may also affect the performance of shortcut activation since participants may find more difficulty when pressing or releasing the shortcut keys with different hands (dominant or non-dominant hands). In the experiment we need the participants to activate the shortcuts at three different locations (namely, dominant-side, middle and non-dominate-side, see Figure 3 left), in order to discover how the location factor affects the activation performance.

4.1 Apparatus

Figure 4: The keyboards used in the Experiment 1. (Top) Apple A1243 Keyboard, (Middle) Logitech G105 keyboard and (Bottom) Filco Majestouch 2 (with blue switch).

Keyboard	Apple	Logitech	Filco
Key sizes	16mm	12mm	12mm
Separation distance	3mm	6mm	6mm
Travel distances	1.5mm	3mm	4mm
Switch type	Membrane	Membrane	Mechanical

Table 1: The shortcuts used in Experiment 3.

The experiment was conducted on a desktop computer, with a 22-inch display running MS Windows 8. Three different keyboards were tested, namely, Apple A1243 Keyboard, Logitech G105, and Filco Majestouch 2 (with blue switch) (Figure 4). All three keyboards are standard 104-keys keyboards with US QWERTY layout and were connected via USB. The key sizes, key separation distances, and travel distances are listed in Table 1. The keyboard being tested were placed on the table of height 70cm, that participants were free to place the keyboard in any location on the table. The experimental application was implemented using C# and .Net Framework.

4.2 Participants

Twelve participants (7 males and 5 females) between the age of 19 to 35 (average 25.2) were recruited. They were mainly undergraduate and graduate students. All had experience with computers and typing, and happened to be right handed. All participants were recruited from a university (recruitment posters were posted for open recruitment). Each of them was given a gift voucher of ∼USD13 for participating in the user study.

4.3 Procedure and Design

The testing application was placed in the middle of the screen with resolution of 500×420. At the beginning of each task, the shortcut key combination was displayed at the top part of the window which indicates the keys of the shortcut (see Figure 3 right). Participants were required to activate the displayed shortcuts by using the instructed technique (DC or UC shortcut), with any hand

and fingers. The displayed keys will also blink concurrently (for DC shortcuts) or sequentially (for UC shortcuts) to provide visual hints about which shortcut technique was using.

A textbox was placed at the bottom of the window which shows the keys currently pressed by the participants. Note that the textbox was focused automatically, thus participants were only needed to type the shortcut keys without any other extra operation. If the correct keys were pressed or released (this depends on the shortcut technique being tested was DC or UC), the activation task was considered as completed. The *pressing time* (i.e. the duration between the first and last key-down events) and the *releasing time* (i.e. the duration between the first and last key-up events) were measured for each completed task. If participant pressed any wrong key, no data will be recorded and the participant was needed to redo the task. This means that there was no failure case and we only considered the duration of key up/down events of the completed tasks.

Our experiment was a $2\times3\times3\times3$ within-participant design with the following factors: (1) **Tech**: two shortcut techniques: DC shortcut and UC shortcut (2) **Key**: Three numbers of keys: 2, 3 and 4. (3) **Location**: Three positions of the shortcuts: dominate-side, middle and non-dominate-side. (4) **Distance**: Three keyboards with different travel distances: shallow (Apple keyboard), middle (X) and deep (Filco Majestouch 2).

Each participant performed the experiment in 3 sessions of different travel **Distance** (using the corresponding keyboards); each session was divided into 2 groups of different shortcut techniques. Each **Tech** group was further divided into 6 subgroups of different combinations of **Key** and **Location**. Within each subgroup, each participant needed to complete 10 activation tasks. In the experiment, **Tech** was counterbalanced, **Distance** was ordered in a balanced Latin-square, and the order of subgroups (**Key** and **Location** factors) was randomized. A total of 6480 activation tasks were included in the experiment. Before using each shortcut technique and keyboard, participants were instructed and given a 5 – 10 minute warm-up session to get familiar with the technique and the feedback of keyboard.

4.4 Results and Discussion

4.4.1 Pressing Time

As DC and UC techniques require user to press the shortcut key differently, it is expected that the there is a significant difference of pressing time on **Tech**. The mean pressing time was 21.2ms for DC shortcuts and 247.9ms for UC shortcuts. There is no significant effect for **Location** ($F_{(2,33)}$=1.06, p=0.36; $F_{(2,33)}$=0.33, p=0.72) and **Distance** ($F_{(2,33)}$=2.37, p=0.11; $F_{(2,33)}$=0.14, p=0.87) on both DC and UC techniques, respectively. Figure 5 (left and middle) shows the mean pressing times of both techniques grouped by **Location** and **Distance**. We observed that the DC shortcuts could be activated easily ($< 50ms$) at different locations on the keyboard and with keyboards of different travel distances. Participants also reported that they found no difference or difficulty in using the DC shortcuts at any location with all three tested keyboards.

Repeated measures analysis shows significant effects for **Key** ($F(2,33)$=12.3, p<0.0001; $F(2,33)$=38.2, p<0.0001) on both DC and UC techniques, respectively. Figure 5 (right) shows the mean pressing times of both techniques grouped by **Key**. Recall that the interface design of UC shortcut requires the shortcut keys to be pressed non-simultaneously, thus using more shortcut keys will always give longer pressing time. For the DC shortcuts, we also observed a similar trend of linear increasing of pressing time. This may be due to the inconsistent pressing motions of multiple fingers, and using more keys would lead to a larger inconsistence. Nevertheless, the number of keys was not affecting the performance much as almost all of the activation tasks with DC technique can be completed within very short duration ($< 50ms$).

	Mean Pressing Time (ms)	Standard Deviation (ms)	Suggested Time Threshold t_{DC} (ms)
2-key	10.3	5.59	**21.5**
3-key	18.0	11.3	**40.6**
4-key	35.3	16.9	**69.1**

Table 2: (Left and Middle) Mean and standard deviation of the pressing times for DC shortcut technique grouped by the Key factor. (Right) The suggested time thresholds for activating DC shortcuts.

	Mean Pressing Time (ms)	Standard Deviation (ms)	Suggested Time Threshold t_{DK1} (ms)
2-key	97.9	31.9	**161.7** (t_{DK2})
3-key	225.8	77.3	**380.4**
4-key	419.8	125.2	**670.2**

Table 3: (Left and Middle) Mean and standard deviation of the pressing times for UC shortcut technique grouped by the Key factor. (Right) The suggested time thresholds for dispatching key pressed events.

4.4.2 Suggested Time Thresholds (t_{DC}, t_{DK1} and t_{DK2})

Table 2 shows the mean μ_{DC} and standard deviation σ_{DC} of the pressing times of DC shortcut technique grouped by the **Key** factor. In order to ensure in most cases the simultaneous key pressing can be successfully recognized for activating DC shortcuts, we suggest using the value $\mu_{DC} + 2\sigma_{DC}$ for the time threshold t_{DC} (the exact value should depend on the number of keys used for the DC shortcuts), and expect around 95% of the activations will be recognized.

Table 3 shows the mean μ_{UC} and standard deviation σ_{UC} of the pressing times of UC shortcut technique grouped by the **Key** factor. Recall that the dispatch time thresholds t_{DK1} and t_{DK2} are used for dispatching key pressed events that are not involved in any DC or UC shortcuts (see the dispatch rule 2 and 3 in Listing 1). We suggest using the pressing time of UC shortcut technique as reference for designing the thresholds, namely using the value $\mu_{UC} + 2\sigma_{UC}$ for t_{DK1} (the exact value should depend on the maximum number of keys used for the UC shortcuts), and using the value 161.7ms for t_{DK2}. We expect most UC shortcuts could be activated without incorrectly dispatching any pressed shortcut keys.

Note that in typical typing scenarios most of the general pressing events will be dispatched using dispatch rule 1 and 3 (see discussion of Experiment 2), therefore there will be only small influence for using large value for threshold t_{DK1}. In fact, the dispatch rule 2 will only be involved if users type multiple adjacent keys quickly and it is rare in usual typing scenarios.

4.4.3 Releasing Time

It is interesting that there was also a significant difference of releasing time on **Tech**, with the mean pressing times 15.9ms and 58.0ms for DC and UC shortcuts respectively. The relative long releasing time for the UC shortcuts may be because of the non-synchronous pressings of different fingers, that cause the key releasing to be less consistent then those of DC shortcuts and lead to a longer releasing time.

Figure 6 shows the mean releasing times of both techniques grouped by **Location**, **Distance** and **Key**. There is no significant effect

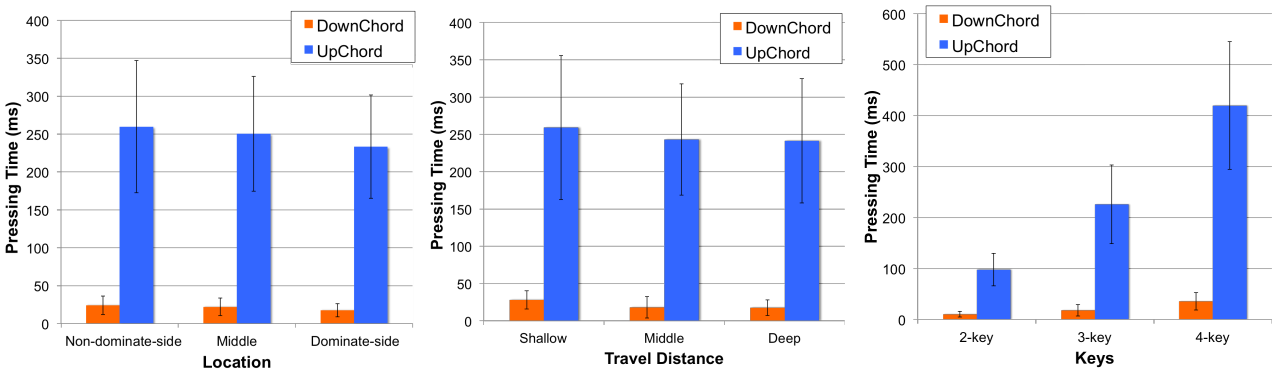

Figure 5: The mean pressing times of shortcut techniques grouped by Location (left), travel Distance (middle) and Key (Right). The error bars represent intervals of one standard deviation.

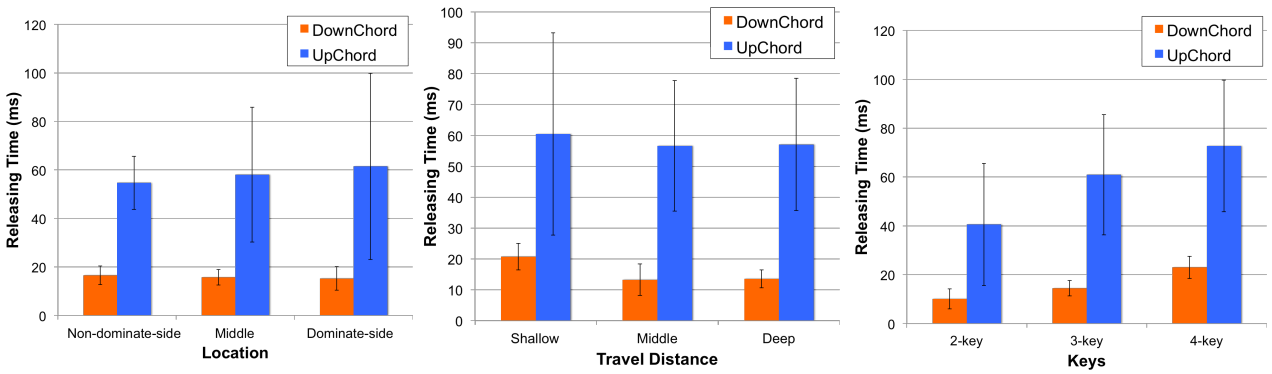

Figure 6: The mean releasing times of shortcut techniques grouped by Location (left), travel Distance (middle) and Key (Right). The error bars represent intervals of one standard deviation.

for **Location** ($F_{(2,33)}$=0.32, p=0.73; $F_{(2,33)}$=0.16, p=0.85) on both DC and UC techniques, respectively. However, we observed that one of the participants cannot release finger simultaneously after performing finger drumming with his dominate hand and give much longer releasing time for UC shortcuts then other participants. This reveals that some users may not able to master the UC activating motion (i.e. finger drumming and then simultaneous releasing) after only a short training period.

Repeated measures analysis shows a significant effect for **Distance** on DC technique ($F_{(2,33)}$=11.1, p=0.00021), this may due to the different switch hardware used of the tested keyboards such that the membrane of the shallow keyboard retains a longer contact time to short circuit the wires after users release the keys. Nevertheless, the releasing times are still short and they are not important to the recognition of DC shortcuts. Note that there is no significant effect for **Distances** on UC shortcut techniques.

	Mean Pressing Time (ms)	Standard Deviation (ms)	Suggested Time Threshold t_{UC} (ms)
2-key	40.6	25.0	**90.0**
3-key	60.9	24.6	**110.1**
4-key	72.7	26.9	**126.5**

Table 4: (Left and Middle) Mean and standard deviation of the releasing times for UC shortcut technique grouped by the Key factor. (Right) The suggested time thresholds for activating UC shortcuts.

4.4.4 Suggested Time Threshold (t_{UC})

Table 4 shows the mean M_{UC} and standard deviation Σ_{UC} of the releasing times of UC shortcut technique grouped by the **Key** factor. In order to ensure in most cases, the simultaneous key releasing can be successfully recognized for activating UC shortcuts, we suggest using the value $M_{UC} + 2\Sigma_{UC}$ for the time threshold t_{UC} (the exact value should depend on the number of keys used for the UC shortcuts), and expect around 95% of the activations will be recognized successfully.

5. EXPERIMENT 2 – INTEGRATION STUDY

After determining both of the activation time thresholds and dispatching time thresholds, we conducted another experiment to examine the integration of the DC and UC shortcut techniques and evaluate their influence in actual typing scenarios.

5.1 Apparatus

Since there is no significant effect in using different keyboards as shown in the result of Experiment 1, in this experiment we only used the keyboard with the middle level of travel distance (i.e. the Logitech model). All other apparatus configuration was same as Experiment 1.

5.2 Participants

6 participants (4 males and 2 females) between the age of 19 to 28 (average 22.2) were recruited. They were mainly undergraduate and graduate students and none of them were involved in the

Experiment 1 and Experiment 3. All participants had experience with computers and typing, and happened to be right handed. The recruit process was same as those of the Experiment 1.

5.3 Procedure and Design

We developed a simple text editor application for this experiment and the editor will recognize the DC and UC activations with the suggested time thresholds. The editor looks like the built-in notepad application of Windows OS, but it suppresses the key events involved in any recognized activations, and will not perform any action. Participants was asked to type three different plain text documents (in total 369 words or 1777 non-space characters), and the editor will record the number of unintended shortcut activations, as well as the delay time of each dispatched key press event (i.e. the time difference between the key down time and the dispatch time). Participants were instructed to complete the typing task as naturally as possible, with their natural typing speed and habit. Before starting the test, participants were instructed and given a 5 minute warm-up session to get familiar with the keyboard.

	2-key	3-key	4-key
DC	5	0	0
UC	185	0	0

Table 5: Count of unintended activations in Experiment 2 grouped by shortcut techniques.

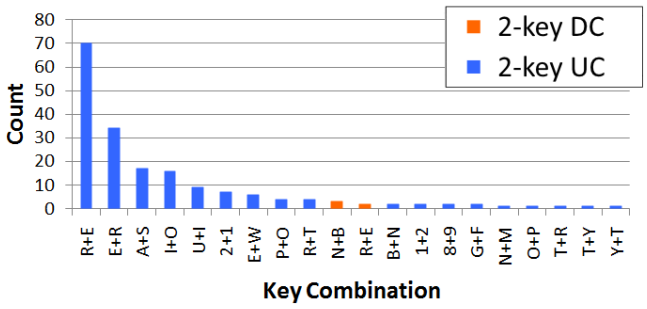

Figure 7: Count of unintended activations in Experiment 2 grouped by key combination.

	Rule 1	Rule 2	Rule 3
Count	2714	6	9891
Mean Delay (ms)	132.3	675.3	178.9

Table 6: Count and mean delay of dispatched key press events grouped by dispatch rules.

5.4 Results and Discussion

Table 5 shows the total unintended activations of all participants grouped by the shortcut techniques, and Figure 7 shows the counts of their key combinations. As shown in the result more than 90% of the unintended activations were of type 2-Key UC, which was mainly due to the fast typing of the adjacent letters of words. Therefore, we suggest not using the 2-key UC shortcuts with regular alphabet keys to avoid such unintended shortcut activations, while it is still possible to use the 2-key UC shortcuts with other keys like function keys since these keys are less frequently being typed sequentially.

Table 6 shows the total count and mean delay of all dispatched key press events grouped by the dispatch rules. It is shown that almost most of the key press events were dispatched by the rule 1 and 3, and the average dispatch time of all dispatched key press events were 169.1ms, which indicates that there was only small latency introduced with the integration of shortcut interface. Some participants also reported that the typing delay was just noticeable when typing in fast speed, however the delay was still acceptable and did not affect the input much.

6. EXPERIMENT 3 – PERFORMANCE COMPARISON

To quantitatively evaluate the performance of the DC and UC shortcuts in actual typing scenarios, we conducted the third experiment for comparing the performance of our shortcut technique with the traditional modifier-based shortcuts.

6.1 Apparatus

Same as those in Experiment 2.

6.2 Participants

6 male and 6 female participants between the age of 18 to 35 (average 25.9) were recruited for the experiment. They were mainly undergraduate and graduate students and five of them were involved in the Experiment 1. All participants had experience with computers and typing, and happened to be right handed. The recruit process was same as those of the Experiment 1.

6.3 Procedure and Design

Participants needed to type and stylize three different documents that containing a similar set of formatting properties (including text color, text size, text alignment and subscript/superscript). We chose these formatting properties rather than the other more common ones (e.g. bold, italic, and underline) since we do not want bias toward the existing modifier-based shortcuts as many participants could already familiar with them. Even though many of the chosen formatting properties could be specified with modifier-based shortcuts in some common commercial text editors, most of the participants reported that they did not know those modifier-based shortcuts and need to refer the shortcut tables during the experiment.

2-key DC	3-key DC / UC	Modifier-based	Format	Description
1+2	1+2+3	Ctrl + 1	ABC	Red text color
5+6	4+5+6	Ctrl + 2	ABC	Green text color
9+0	8+9+0	Ctrl + 3	ABC	Blue text color
T+Y	R+T+Y	Ctrl + 0	ABC	Black text color
Q+W	Q+W+E	Ctrl + '<'	ABC	Reduce font size
O+P	I+O+P	Ctrl + '>'	ABC	Increase font size
A+S	A+S+D	Ctrl + 'L'	ABC	Align text left
F+G	F+G+H	Ctrl + 'E'	ABC	Center text
K+L	J+K+L	Ctrl + 'R'	ABC	Align text right
Z+X	Z+X+C	Ctrl + '-'	A_{BC}	Subscript text
N+M	B+N+M	Ctrl + '+'	A^{BC}	Superscript text

Table 7: The shortcuts used in Experiment 3.

Table 7 lists the shortcuts used for the experiment, the modifier-based shortcuts were referred from the corresponding shortcuts of Microsoft Word (except the text color shortcuts which is not defined by default). Also note that participants can activate the modifier-based shortcuts with single- or both-hand typing, which means they

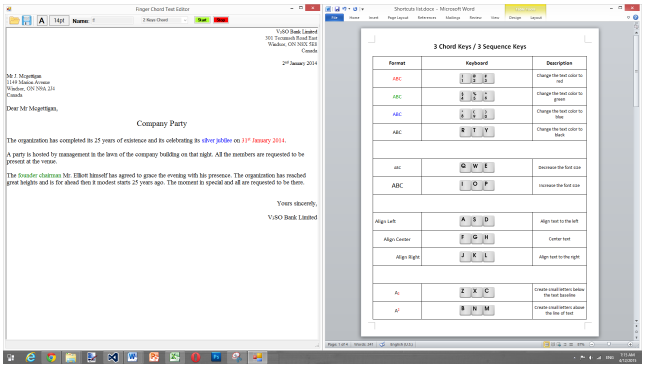

Figure 8: (Left) Screen capture of the testing program used for Experiment 3. (Right) Shortcut list of the testing technique shown to the participants.

can use their familiar and comfortable typing poses during the test. Each documents included all the formats listed in the table however the order and number of the shortcuts involved were depended on the participants' typing preferences, for example one can finish typing the whole text first before applying any formatting property, and participant can select multiple words or lines and apply a formatting property to the whole selection with one shortcut operation. Note that participants were only needed to press the corresponding keys but not typing the exact character shown in the table (e.g. no shift key was used when typing the '<' key). We consider using only the 2-key DC, 3-key DC, 3-key UC and modifier-based shortcut techniques in the experiment because we would like to ensure each participant could finish the assigned tasks within a reasonable time and avoid degraded performance due to fatigue. The testing application only accepts backspace key as the only way to delete unwanted text (the delete key and undo / redo shortcuts were disabled), since we would like to use the number of backspace key presses as the measurement of the number of corrections of the typing task.

The testing application, as a simple text editor, was placed at the left part of the display and the shortcut list of the testing technique was displayed at the right, as shown in Figure 8. The printed reference document was given to participants in the beginning of the task. Each participant performed the experiment in 4 sessions of different shortcut techniques; within each session participants were asked to type and stylize three documents with the tested technique. The techniques were ordered in a balanced Latin-square and the order of assigned documents was randomized. Participants were instructed to complete the typing task as naturally as possible, with their natural typing speed and habit; also they can activate the shortcut using any hand or fingers with the provided shortcut key combination. We recorded all the key press and release events and measured the numbers of all shortcut used, complete time of each task, number of backspace pressed and the number of failure shortcut activations. The number of failure shortcut activations was measured by manually checking the recorded key events to figure out any incorrectly dispatched key should be involved in an unsuccessful shortcut activation. Before using each shortcut technique, participants were instructed and given a 5-10 minute warm-up session to get familiar with the technique and the shortcuts.

6.4 Results and Discussion

Figure 9 shows the total number of shortcuts used, the total complete time, the total number of backspace key pressed and the total

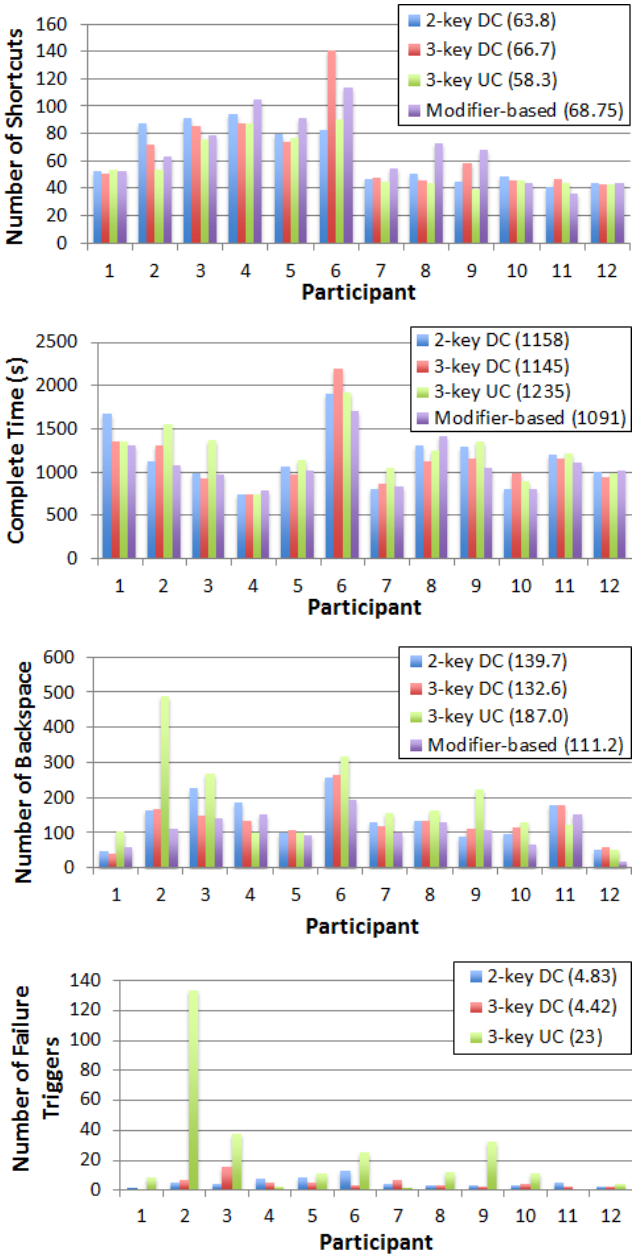

Figure 9: The performance measurements of each participant: (From top to bottom) total number of shortcut used, total complete time, total number of backspace key pressed and total number failure activations.

number of failure shortcut activations by each participant. Also the mean values are shown in the legends. There was no significant effect for the number of shortcuts used ($F_{(3,44)} = 0.439, p = 0.726$), the total complete time ($F_{(3,44)} = 0.389, p = 0.761$), the number of backspace pressed ($F_{(3,44)} = 1.95, p = 0.135$), and the number of failure activations ($F_{(2,33)} = 2.95, p = 0.066$).

The results support the assumption that our tested shortcut techniques (namely, 2-key and 3-key DC, and 3-key UC shortcuts) are potential effective alternative shortcut techniques to the traditional modifier-based shortcuts, as they gave competitive performance in the aspects of number of shortcuts used and complete time. How-

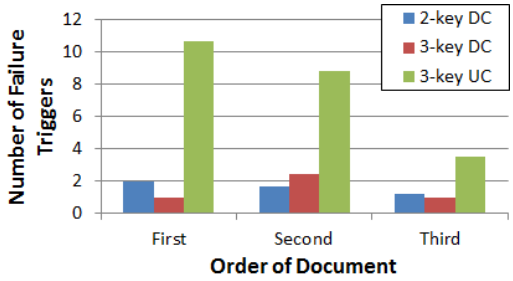

Figure 10: The mean number of failure shortcut activations for the three documents grouped by the shortcut techniques.

ever, the results also show that the 3-key UC technique has the highest mean correction count (number of backspace pressed) and mean failure activation count, indicating that this is the most difficult shortcut technique among the tested methods. Particularly we observed that participant 2 cannot successfully activate the 3-key UC shortcuts in the beginning of the test, thus leaded to large amount of unintended key input and unwanted corrections. This shows that a few users may not able to master the UC shortcut activation with only short duration training. Figure 10 shows the mean number of failure activations of all participants for the three documents grouped by shortcut techniques, and is used for discovering any learning effect by comparing the changes of the failure counts throughout the 3 typing tasks assigned to the participants. Note that there a large learning effect for the 3-key UC technique and in fact participant 2 had an obvious improvement in using the 3-key UC shortcuts from having 85 failure activations during typing the first document to only 9 failure activations for the third document. We believe that with a longer training time users can give a better performance when using the UC shortcut technique.

7. CONCLUSION

Typical typing, as well as many other keyboard operations including using modifier-based shortcuts, are dependent upon the order of key press and avoid simultaneous key pressing. This paper introduces DownChord and UpChord shortcuts, the new keyboard shortcuts techniques, which utilizes the simultaneous key pressing and releasing for defining new classes of shortcuts. Our technique gives natural extension of the design space of keyboard shortcut, allows introduction of more shortcuts and enriches keyboard interaction, which is useful in different applications, particularly those with large set of predefined operations such as professional 3D software. The new shortcuts can be used seamlessly with typical typing and existing modifier-based shortcuts. We conducted controlled experiments to determine the appropriate time thresholds to allow our shortcuts to be activated effortlessly without long undesired delay between key pressing and character display. This leads to a highly integrated and controllable command input interface. Results of our user study show that both DownChord and UpChord shortcuts are promising shortcut techniques which gives competitive performance with modifier-base shortcuts.

7.1 Limitations

Because of the keyboard ghosting problem which mostly due to the cost-down of circuit design in low-cost keyboards, simultaneous key pressing with three of more keys on multiple rows and columns may not be detected correctly. This limits the design space of the new shortcuts as shortcuts are only possible to be defined on the same row or column.

To most of the users the UC shortcuts are kind of complete new typing operation, we expect these shortcuts have a lower acceptability and users may need a relative long training to master the technique. Therefore, we suggest associating less frequently used commands to UC shortcuts while more common commands should be associated to the DC shortcuts or traditional modifier-based shortcuts.

7.2 Future Work

As a future work, we would like to apply the concept of our shortcut techniques to shortcuts that involve multiple keys at separated locations or use both hand operations, in order to provide a richer design space and shortcuts with better spatial association. We would like to investigate the possibility of applying our technique for single hand text input by using DC or UC techniques for inputting part of the alphabets. We believe it could potentially reduce the total number of required keys and improve typing performance, comparing with exist key-chord typing methods. We would also like to apply and evaluate our shortcut techniques in other keyboard types and application scenarios, such as multitouch input devices and graphic object manipulation.

8. ACKNOWLEDGMENTS

We thank all the reviewers of this paper for the insightful comments and constructive suggestions. The work described in this paper was supported by the grant from City University of Hong Kong (CityU Reference No.: 7004498).

9. REFERENCES

[1] Cykey keyboard, bellaire electronics. http://www.cykey.co.uk/. Accessed: 2015-04-14.

[2] Keyboard ghosting explained! http://www.microsoft.com/appliedsciences/antighostingexplained.mspx. Accessed: 2015-04-14.

[3] "microwriter". old computers.com. http://www.old-computers.com/museum/computer.asp?c=558&st=1. Accessed: 2015-04-14.

[4] Twiddler 3. http://twiddler.tekgear.com/. Accessed: 2015-04-14.

[5] O. K.-C. Au and C.-L. Tai. Multitouch finger registration and its applications. In *Proceedings of the 22Nd Conference of the Computer-Human Interaction Special Interest Group of Australia on Computer-Human Interaction*, OZCHI '10, pages 41–48, New York, NY, USA, 2010. ACM.

[6] G. Bailly, T. Pietrzak, J. Deber, and D. J. Wigdor. Métamorphe: Augmenting hotkey usage with actuated keys. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems*, CHI '13, pages 563–572, New York, NY, USA, 2013. ACM.

[7] F. Block, H. Gellersen, and N. Villar. Touch-display keyboards: Transforming keyboards into interactive surfaces. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems*, CHI '10, pages 1145–1154, New York, NY, USA, 2010. ACM.

[8] M. N. Bonner, J. T. Brudvik, G. D. Abowd, and W. K. Edwards. No-look notes: Accessible eyes-free multi-touch text entry. In *Proceedings of the 8th International Conference on Pervasive Computing*, Pervasive'10, pages 409–426, Berlin, Heidelberg, 2010. Springer-Verlag.

[9] P. H. Dietz, B. Eidelson, J. Westhues, and S. Bathiche. A practical pressure sensitive computer keyboard. In *Proceedings of the 22Nd Annual ACM Symposium on User*

Interface Software and Technology, UIST '09, pages 55–58, New York, NY, USA, 2009. ACM.

[10] L. Findlater, B. Lee, and J. Wobbrock. Beyond qwerty: Augmenting touch screen keyboards with multi-touch gestures for non-alphanumeric input. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems*, CHI '12, pages 2679–2682, New York, NY, USA, 2012. ACM.

[11] J. Jannotti. Iconic text entry using a numeric keypad. In *Unpublished. http://www.jannotti.com/papers/iconic-uist02.pdf*, 2002.

[12] L. Sholes. Improvement in type-writing machines, Aug. 27 1878. US Patent 207,559.

[13] S. Taylor, C. Keskin, O. Hilliges, S. Izadi, and J. Helmes. Type-hover-swipe in 96 bytes: A motion sensing mechanical keyboard. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems*, CHI '14, pages 1695–1704, New York, NY, USA, 2014. ACM.

[14] J. Wagner, E. Lecolinet, and T. Selker. Multi-finger chords for hand-held tablets: Recognizable and memorable. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems*, CHI '14, pages 2883–2892, New York, NY, USA, 2014. ACM.

[15] J. Wobbrock, B. Myers, and B. Rothrock. Few-key text entry revisited: Mnemonic gestures on four keys. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems*, CHI '06, pages 489–492, New York, NY, USA, 2006. ACM.

[16] S. Zhai, P. O. Kristensson, P. Gong, M. Greiner, S. A. Peng, L. M. Liu, and A. Dunnigan. Shapewriter on the iphone: From the laboratory to the real world. In *CHI '09 Extended Abstracts on Human Factors in Computing Systems*, CHI EA '09, pages 2667–2670, New York, NY, USA, 2009. ACM.

[17] H. Zhang and Y. Li. Gestkeyboard: Enabling gesture-based interaction on ordinary physical keyboard. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems*, CHI '14, pages 1675–1684, New York, NY, USA, 2014. ACM.